

*Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique*



*Université Polytechnique de
Bingerville*

Année Académique : 2025-2026

MASTER 1

DATA-IA

MODULE: NoSQL

**RAPPORT DE MISE EN ŒUVRE D'UN DATA
LAKE MONGODB DE
DÉTECTION DE FRAUDE À PARTIR DE FLUX
TRANSACTIONNELS MULTI-SOURCES**

Réalisé par :

YAO MIÉZAN SAM WILLIAM

Enseignant :

M. SORO SEYDOU

Sommaire

Sommaire.....	i
Liste des figures.....	ii
Liste des tableaux	iii
1. Contexte et objectifs	1
2. Sources de données	1
2.1 Flux Mobile Money	1
2.2 Flux Ventes Commerce	1
2.3 Qualité initiale des données	1
3. Architecture du Data Lake	2
4. Méthodologie.....	3
4.1 Couche Bronze, ingestion brute	3
4.2 Couche Silver, nettoyage et structuration	4
4.3 Couche Gold, features de détection de fraude	5
4.4 Règles de scoring et alertes	6
5. Résultats	6
5.1 Volumétrie par couche	6
5.2 Alertes de fraude générées.....	7
5.3 Performance et passage à l'échelle	7
6. Limites et points d'attention	8
7. Conclusion.....	8
Bibliographie.....	9
Annexes : code des différentes couches	10

Liste des figures

Figure 1 : Architecture fonctionnelle du Data Lake de détection de fraude basé sur MongoDB	2
Figure 2 : Téléchargement de MongoDB Database Tools	3
Figure 3 : Fin de l'installation de MongoDB Database Tools	3
Figure 4 : Ingestion brute des deux flux via mongoimport	4
Figure 5 : Vérification du chargement bronze dans mongosh	4
Figure 6 : Résultat de la transformation Silver, flux Mobile Money et Ventes	5
Figure 7 : Création des index sur les collections Silver	5
Figure 8 : Application du validateur JSON Schema et test de rejet d'un document incomplet ..	5
Figure 9 : Résultat du calcul des features de fraude, couche Gold	6
Figure 10 : Résultat final des règles de scoring, avec niveaux de sévérité	7

Liste des tableaux

<i>Tableau 1 : Architecture du Data Lake en couches</i>	2
<i>Tableau 2 : Volumétrie des collections par couche</i>	6
<i>Tableau 3 : Répartition des alertes de fraude par niveau de sévérité</i>	7

1. Contexte et objectifs

Ce projet met en place un Data Lake sur MongoDB [1] pour détecter des transactions suspectes en croisant deux flux transactionnels indépendants circulant en Côte d'Ivoire : un flux de transactions Mobile Money et un flux de ventes de commerçants à Abidjan. L'objectif est de démontrer comment un modèle de données documentaire, flexible et adapté aux sources hétérogènes, permet de construire un pipeline de détection de fraude explicable, depuis l'ingestion des données brutes jusqu'à la génération d'alertes. Les systèmes de détection de fraude reposent sur l'identification de comportements inhabituels à partir des transactions observées afin de limiter les pertes financières et les activités frauduleuses [2].

Le sujet impose une approche multi-sources. Les deux flux ont des schémas différents, proviennent de systèmes distincts, mais se recoupent sur une période commune (janvier à juin 2024) et des zones géographiques communes (Adjamé, Cocody, Marcory, Plateau, Yopougon), ce qui permet une véritable corrélation entre sources plutôt qu'une simple analyse isolée.

2. Sources de données

2.1 Flux Mobile Money

Fichier transactions_mobile_money_100k.csv, 100 000 transactions réparties sur 4 opérateurs (MTN CI, Orange Money, Moov Africa, Wave). Chaque transaction comporte un identifiant, une date et heure, un opérateur, un type d'opération (dépôt, retrait, transfert, paiement marchand, recharge), un expéditeur, un bénéficiaire, un montant, des frais, une zone d'expéditeur et de bénéficiaire, un agent, et un statut.

2.2 Flux Ventes Commerce

Fichier ventes_commerce_abidjan_100k.csv, 100 000 ventes effectuées par des commerçants abidjanais, dont 33 % réglées en Mobile Money. Chaque vente comporte un produit, une quantité, un prix unitaire, un vendeur, une zone client, un mode de paiement et un montant.

2.3 Qualité initiale des données

Une analyse préalable a permis d'identifier plusieurs anomalies conservées volontairement lors de l'ingestion brute, conformément au principe d'un Data Lake.

- 200 transactions Mobile Money avec un montant inférieur ou égal à 0 (anomalie évidente)
- 3 000 transactions Mobile Money sans identifiant d'agent renseigné
- 5 000 ventes sans vendeur renseigné
- Aucune incohérence entre quantité multipliée par prix unitaire et montant total côté ventes (donnée cohérente)

3. Architecture du Data Lake

L'architecture retenue suit le modèle en couches de la Medallion Architecture [3], inspiré des principes des Data Lakes [4], qui consiste à organiser les données en plusieurs niveaux afin de conserver les données brutes tout en appliquant progressivement les transformations nécessaires. Cette approche sépare clairement les responsabilités de chaque étape du pipeline et garantit une traçabilité complète des traitements, depuis l'ingestion jusqu'à la génération des alertes.

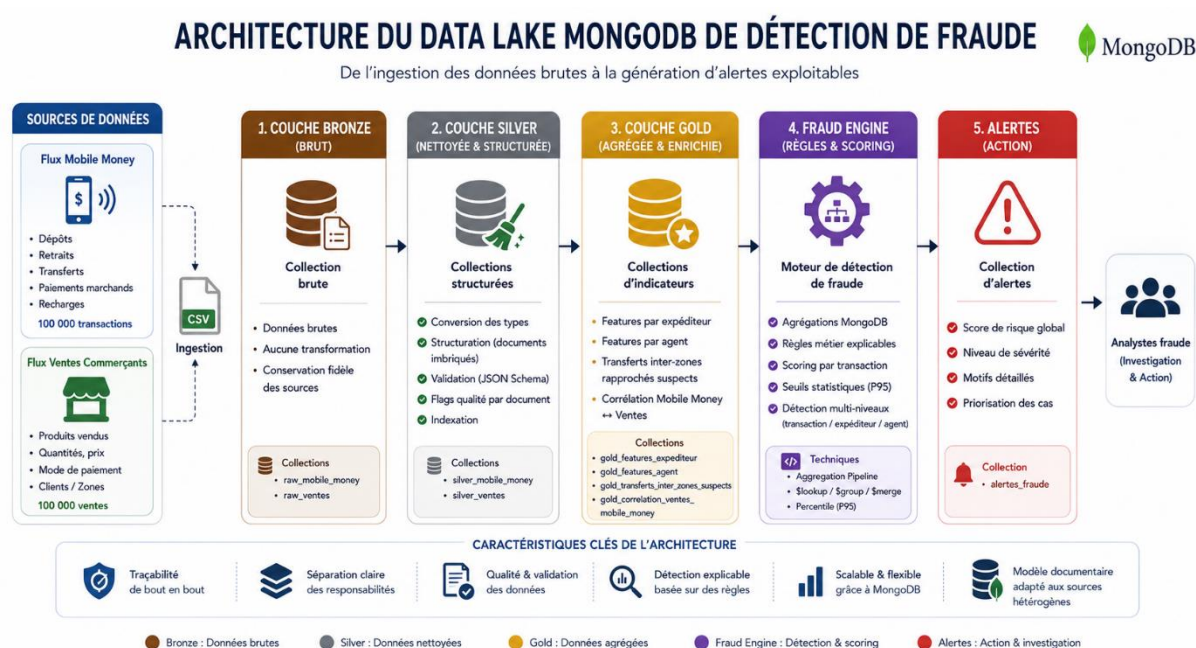


Figure 1 : Architecture fonctionnelle du Data Lake de détection de fraude basé sur MongoDB.

Couche	Collections	Rôle
Bronze	raw_mobile_money, raw_ventes	Copie brute des CSV, aucune transformation
Silver	silver_mobile_money, silver_ventes	Typage, structuration en documents imbriqués, flags qualité
Gold	gold_features_expéditeur, gold_features_agent, gold_transferts_inter_zones_suspects, gold_correlation_ventes_mobile_money	Indicateurs agrégés par entité
Alertes	alertes_fraude	Transactions identifiées avec motif, score et niveau de sévérité

Tableau 1 : Architecture du Data Lake en couches.

4. Méthodologie

4.1 Couche Bronze, ingestion brute

Les deux fichiers CSV sont chargés tels quels via mongoimport [5], sans transformation, afin de conserver une copie fidèle et intégrale de la donnée source, y compris ses défauts. Ce principe garantit qu'aucune information n'est perdue avant l'étape de nettoyage, et permet de revenir en arrière à tout moment. L'outil mongoimport fait partie des MongoDB Database Tools, un ensemble de programmes en ligne de commande distinct du serveur et de mongosh, qu'il faut installer séparément.

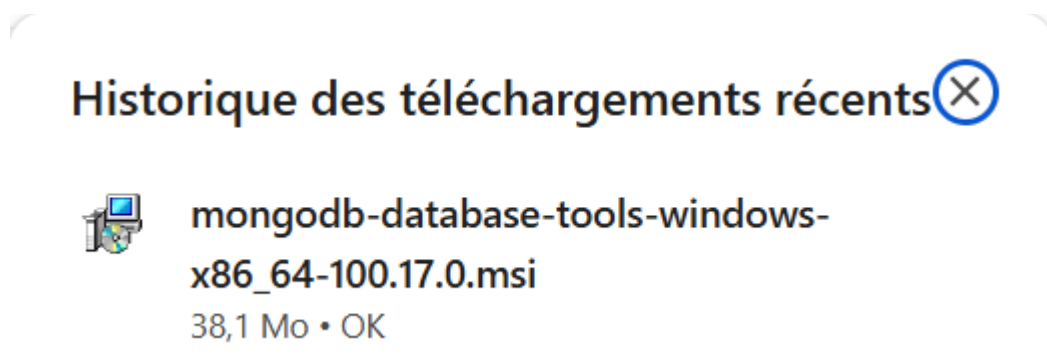


Figure 2 : Téléchargement de MongoDB Database Tools.

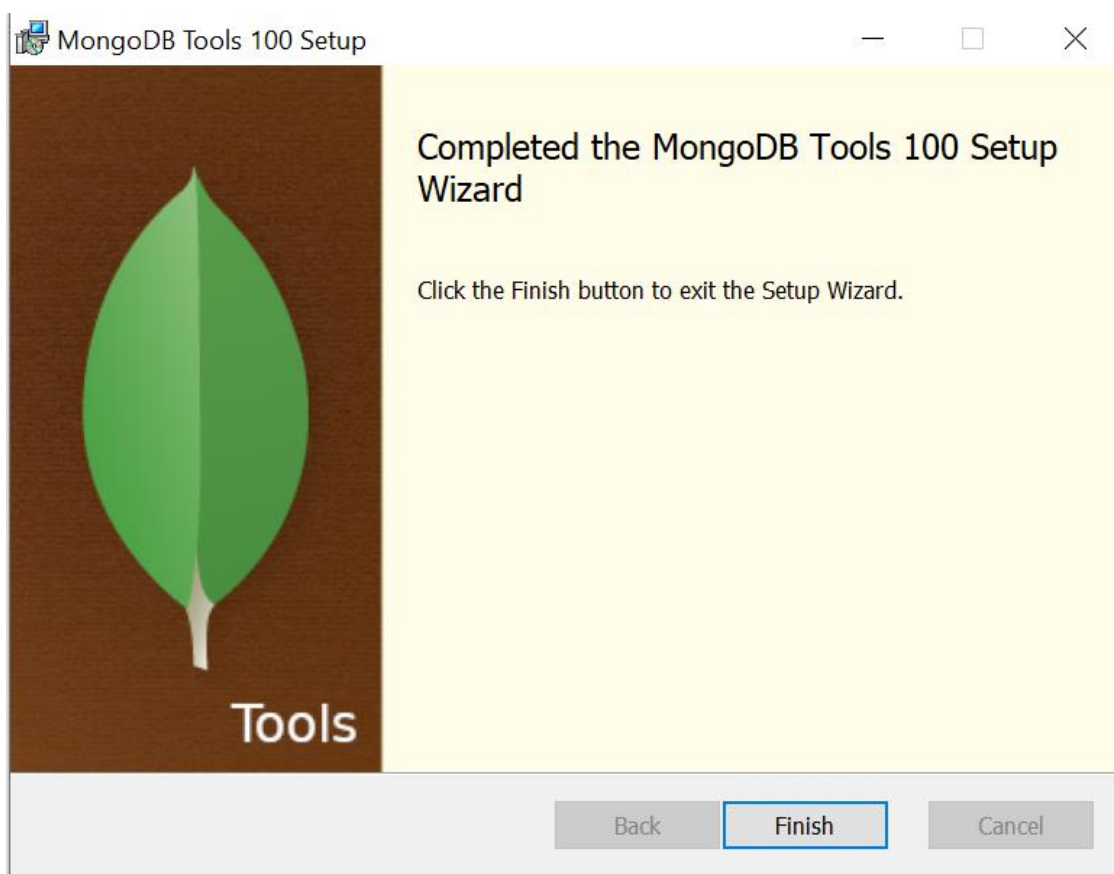


Figure 3 : Fin de l'installation de MongoDB Database Tools.

```

Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\MIEZANSAM>mongoimport --version
mongoimport version: 100.17.0
git version: 0b142f65e139525881b6a343bc56d8d98e5a1f90
go version: go1.25.9
os: windows
arch: amd64
compiler: gc

C:\Users\MIEZANSAM>mongoimport --db fraud_dataLake --collection raw_mobile_money --type csv --headerline --file "C:/Users/MIEZANSAM/Documents/transactions_mobile_money_100k.csv"
connected to: mongodb://localhost/
2026-07-24T14:25:10.484+0000 [#####.....] fraud_dataLake.raw_mobile_money 5.35MB/13.2MB (40.5%)
2026-07-24T14:25:16.404+0000 [#####.....] fraud_dataLake.raw_mobile_money 11.4MB/13.2MB (86.5%)
2026-07-24T14:25:18.522+0000 [#####.....] fraud_dataLake.raw_mobile_money 13.2MB/13.2MB (100.0%)
2026-07-24T14:25:18.522+0000 100000 document(s) imported successfully. 0 document(s) failed to import.

C:\Users\MIEZANSAM>mongoimport --db fraud_dataLake --collection raw_ventes --type csv --headerline --file "C:/Users/MIEZANSAM/Documents/ventes_commerce_abidjan_100k.csv"
connected to: mongodb://localhost/
2026-07-24T14:25:31.452+0000 [#####.....] fraud_dataLake.raw_ventes 5.43MB/7.70MB (70.5%)
2026-07-24T14:25:35.586+0000 [#####.....] fraud_dataLake.raw_ventes 7.70MB/7.70MB (100.0%)
2026-07-24T14:25:35.587+0000 100000 document(s) imported successfully. 0 document(s) failed to import.

C:\Users\MIEZANSAM>

```

Figure 4 : Ingestion brute des deux flux via mongoimport.

```

Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\MIEZANSAM>mongo
Current MongoDB Log ID: 6a637617743fd9e6566e433d
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.9.2
Using MongoDB: 7.0.37
Using Mongosh: 2.9.2

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-07-20T11:02:43.435+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use fraud_dataLake
switched to db fraud_dataLake
fraud_dataLake> db.raw_mobile_money.countDocuments()
100000
fraud_dataLake> db.raw_ventes.countDocuments()
100000
fraud_dataLake> db.raw_mobile_money.findOne()
{
  _id: ObjectId('6a6375c089181a6da35b7ba4'),
  id_transaction: 'M00000001',
  date_heure: '2024-04-13 18:12:00',
  operateur: 'Orange Money',
  type_operation: 'Repet',
  expéditeur: '+225 01 99 77 97',
  bénéficiaire: '+225 25 21 89 72',
  montant_fcfa: 91784,
  frais_fcfa: 734,
  zone_expéditeur: 'Abidjan-Adjamé',
  zone_bénéficiaire: 'Abidjan-Adjamé',
  id_agent: 'A670105',
  statut: 'Succès'
}
fraud_dataLake> db.raw_ventes.findOne()
{
  _id: ObjectId('6a6375db02c43df17520c7dd'),
  id_transaction: 'TX000004',
  date: '2024-06-03',
  produit: 'riz (sac 25kg)',
  quantite: 10,
  prix_unitaire: 3500,
  vendeur: 'Adjoua B.',
  zone_client: 'Marcury',
  mode_paiement: 'Mobile Money',
  fraud_dataLake>

```

Figure 5 : Vérification du chargement bronze dans mongosh.

4.2 Couche Silver, nettoyage et structuration

Un pipeline d'agrégation MongoDB [6] convertit les champs texte du bronze en types corrects (dates, nombres), restructure les documents avec des sous-objets imbriqués (expéditeur, bénéficiaire, vendeur, client), et calcule des indicateurs de qualité par document (montant valide, agent manquant, cohérence du montant). Ces flags sont conservés dans chaque document plutôt que d'être calculés à la volée, ce qui accélère les analyses ultérieures.


```

C:\Users\MIEZANSAM>mongo
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\MIEZANSAM>mongo
Current MongoDB Log ID: 6a637b7ff8f4e75b02711726
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongoshv2.9.2
Using MongoDB: 7.0.37
Using Mongosh: 2.9.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-07-20T11:02:43.435+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use fraud_datalake
switched to db fraud_datalake
fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/02_silver_mobile_money.js")
Transformation silver_mobile_money terminée.
Nombre de documents : 100000
Documents avec montant invalide : 200
Documents avec agent manquant : 3000
true
fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/03_silver_ventes.js")
Transformation silver_ventes terminée.
Nombre de documents : 100000
Documents avec montant incohérent (qte x prix ≠ montant) : 0
Documents avec vendeur manquant : 5000
true

```

Figure 6 : Résultat de la transformation Silver, flux Mobile Money et Ventes.

```

fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/04_indexes.js")
Index créés.
Index silver_mobile_money : ["_id","expediteur.numero_1","beneficiaire.numero_1","idAgent_1","dateHeure_1","expediteur.zone_1_beneficiaire.zone_1","statut_1"]
Index silver_ventes : ["_id","vendeur.nom_1","client.zone_1","date_1","modePaiement_1"]
true
fraud_datalake>

```

Figure 7 : Création des index sur les collections Silver.

4.2.1 Validation de schéma

Au-delà des flags de qualité calculés au moment du chargement, un validateur JSON Schema [7] est appliqué à la collection `silver_mobile_money` via la commande `collMod`, en `validationLevel` `moderate`. Cette validation ne modifie pas les documents déjà présents, mais garantit qu'à partir de son activation, toute nouvelle insertion ou modification respecte la structure attendue (champs obligatoires, types corrects, sous-documents expéditeur et bénéficiaire bien formés). C'est un filet de sécurité complémentaire aux flags de qualité, qui eux documentent l'état de la donnée sans empêcher une future erreur de structure.

```

fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/07_validator_silver.js")
Valdateur JSON Schema appliqué à silver_mobile_money (validationLevel: moderate).
Test réussi : le validateur rejette bien un document incomplet.
Message : Document failed validation
true
fraud_datalake>

```

Figure 8 : Application du validateur JSON Schema et test de rejet d'un document incomplet.

4.3 Couche Gold, features de détection de fraude

Quatre familles d'indicateurs sont calculées par agrégation :

- Profil par expéditeur : nombre de transactions, montant moyen, écart-type des montants, nombre de zones et de bénéficiaires distincts, taux d'échec
- Profil par agent Mobile Money : nombre de transactions traitées, taux d'échec
- Transferts inter-zones rapprochés : détection d'un même expéditeur opérant depuis deux zones différentes en moins de 30 minutes
- Corrélation ventes et Mobile Money : ventes payées en Mobile Money correspondant, par montant, zone et date, à une transaction Mobile Money enregistrée

```

fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/05_gold_features.js")
gold_features_expéditeur créée : 98670 expéditeurs distincts.
gold_features_agent créée : 200 agents distincts.
gold_transferts_inter_zones_suspects créée : 0 expéditeurs concernés.
gold_correlation_ventes_mobile_money créée : 73 ventes corrélées.
true
fraud_datalake> _

```

Figure 9 : Résultat du calcul des features de fraude, couche Gold.

4.4 Règles de scoring et alertes

Un système de règles explicable, et non un modèle de machine learning, attribue un score de risque à chaque transaction, en combinant trois niveaux d'anomalie. La transaction elle-même (montant invalide), l'expéditeur (vélocité, diversité des bénéficiaires) et l'agent (taux d'échec anormal). Les seuils sont calculés statistiquement lorsque cela est pertinent (percentile 95), ou fixés manuellement lorsque la distribution des données rend le percentile inexploitable. Ce point est détaillé en section 6.

Les poids attribués à chaque critère ne sont pas arbitraires, ils reflètent la robustesse de la preuve statistique disponible. Un montant invalide (3 points) et un agent au taux d'échec anormal (3 points) sont les deux signaux les plus lourds, car ce sont des faits directement observables ou calculés sur un échantillon suffisant (au moins 30 transactions pour un agent), sans ambiguïté d'interprétation. La variabilité des montants chez un expéditeur (2 points) combine à la fois une anomalie de fréquence et une anomalie de montant, ce qui en fait un signal plus fort qu'un critère isolé, mais elle reste limitée par la faible profondeur d'historique des expéditeurs dans ce jeu de données. La vélocité et la diversité des bénéficiaires (1 point chacune) sont volontairement les signaux les plus faibles du système, car ils reposent ici sur des expéditeurs n'ayant au maximum que 2 ou 3 transactions sur toute la période, une preuve trop mince pour justifier un poids élevé à elle seule. C'est pourquoi une transaction n'est retenue comme alerte qu'à partir d'un score cumulé de 2, obligeant les signaux faibles à se combiner entre eux ou avec un signal fort pour déclencher une alerte.

5. Résultats

5.1 Volumétrie par couche

Collection	Documents
raw_mobile_money et raw_ventes	100 000 + 100 000
silver_mobile_money et silver_ventes	100 000 + 100 000
gold_features_expéditeur	98 670 expéditeurs distincts
gold_features_agent	200 agents distincts
gold_transferts_inter_zones_suspects	0 cas détecté
gold_correlation_ventes_mobile_money	73 ventes corrélées

Tableau 2 : Volumétrie des collections par couche.

5.2 Alertes de fraude générées

Le système a généré 7 134 alertes sur 100 000 transactions Mobile Money, soit 7,1 % du volume, réparties en trois niveaux de sévérité pour faciliter la priorisation d'une éventuelle investigation.

Niveau	Nombre d'alertes	Interprétation
PRIORITAIRE	134	Plusieurs critères forts combinés, investigation recommandée en premier
SUSPECT	4 480	Un critère fort ou une combinaison de critères faibles
À SURVEILLER	2 520	Signal faible isolé, à confirmer avec d'autres sources

Tableau 3 : Répartition des alertes de fraude par niveau de sévérité.

```

fraud_datalake> load("C:/Users/MIEZANSAM/Documents/scripts/06_fraud_rules.js")
Seuil fixe (justifié) nombre de transactions par expéditeur : 2
Seuil fixe (justifié) nombre de bénéficiaires distincts : 2
Seuil P95 taux d'échec agent (agents avec >= 30 transactions) : 0.122
Expéditeurs à risque (score > 0) : 1315
fraud_datalake> _
-----
Total alertes générées : 7134
Répartition par score :
  score 2 : 2520 alertes
  score 3 : 4480 alertes
  score 5 : 125 alertes
  score 6 : 9 alertes
-----
Répartition par niveau de sévérité :
  SUSPECT : 4480 alertes
  A_SURVEILLER : 2520 alertes
  PRIORITAIRE : 134 alertes
true

```

Figure 10 : Résultat final des règles de scoring, avec niveaux de sévérité.

Le facteur dominant dans la génération des alertes est le risque associé aux agents. 9 agents sur 200, dépassant un taux d'échec de 12,2 % (seuil du 95e percentile, calculé uniquement sur les agents ayant traité au moins 30 transactions pour éviter tout biais d'échantillon), concentrent la majorité des alertes de niveau SUSPECT et PRIORITAIRE.

5.3 Performance et passage à l'échelle

L'ensemble du pipeline a été exécuté sur une instance MongoDB locale unique, sans réplication ni sharding, avec un volume de 100 000 documents par flux, un ordre de grandeur suffisant pour valider l'architecture sans nécessiter d'infrastructure distribuée. L'opération la plus coûteuse du pipeline est la corrélation ventes et Mobile Money (section 4.3), qui effectue une jointure entre les 33 000 ventes réglées en Mobile Money et les 100 000 transactions candidates. Deux index dédiés (montantFcfa et beneficiaire.zone) ont été créés spécifiquement pour supporter cette jointure côté collection cible, afin que MongoDB puisse s'appuyer sur un accès indexé plutôt que sur un parcours complet de la collection à chaque itération. Sur ce volume, l'ensemble des couches Gold s'exécute en quelques secondes. Un passage à plusieurs millions de transactions nécessiterait vraisemblablement un sharding sur la clé expéditrice ou sur la date, ainsi qu'un pré-filtrage plus agressif avant la jointure de corrélation, pour éviter une dégradation du temps de traitement à mesure que le volume croît.

6. Limites et points d'attention

Une analyse honnête des résultats a mis en évidence plusieurs limites qu'il est important de documenter plutôt que de masquer.

La première, et la plus fondamentale, concerne la validation des alertes elles-mêmes. Le jeu de données étant synthétique et généré de façon aléatoire, il n'existe aucune vérité terrain, c'est à dire aucune étiquette confirmant qu'une transaction donnée est réellement frauduleuse ou non. Le système produit des scores de risque statistiquement justifiés et des motifs explicables, mais en l'absence de référence, il est impossible de calculer une précision, un rappel ou un taux de faux positifs sur les 7 134 alertes générées. Cette limite n'est pas propre à ce projet, elle concerne tout système de règles construit sur des données non labellisées. Sur des données réelles, la première étape suivante serait de faire valider manuellement un échantillon d'alertes par des experts métier ou des agents Mobile Money, afin d'estimer un taux de faux positifs réel et d'ajuster les seuils en conséquence.

98,7 % des expéditeurs Mobile Money n'effectuent qu'une seule transaction sur toute la période observée, le maximum observé étant de 3 transactions par expéditeur. Cette distribution quasi uniforme rend inopérants certains indicateurs statistiques classiques. Le percentile 95 du nombre de transactions par expéditeur tombe mécaniquement sur 1, et le taux d'échec calculé sur un échantillon d'une seule transaction vaut nécessairement 0 % ou 100 %, sans valeur diagnostique réelle. Ces deux pièges ont été identifiés et corrigés en fixant des seuils manuels justifiés, et en exigeant un nombre minimal d'observations avant de considérer un indicateur comme fiable (au moins 5 transactions pour la variabilité et le taux d'échec au niveau expéditeur, au moins 30 transactions pour le taux d'échec au niveau agent).

Par ailleurs, aucun transfert inter-zones rapproché n'a été détecté dans ce jeu de données. Ce résultat négatif ne remet pas en cause la validité de la requête, il montre que ce scénario de fraude précis n'est simplement pas représenté dans les données de test, probablement parce qu'elles sont générées de façon synthétique et aléatoire plutôt qu'issues d'un système réel.

Ces limites suggèrent qu'un futur travail sur des données réelles, avec des expéditeurs réellement récurrents, permettrait de réactiver pleinement les indicateurs de vélocité et de variabilité, aujourd'hui sous-exploités faute de répétitions suffisantes dans les données disponibles.

7. Conclusion

Ce projet démontre la faisabilité d'un Data Lake de détection de fraude reposant entièrement sur MongoDB, depuis l'ingestion brute multi-sources jusqu'à la génération d'alertes scorées et explicables. L'architecture en couches (bronze, silver, gold, alertes) offre une traçabilité complète de chaque transformation, et le système de règles, bien que simple, s'appuie sur des seuils statistiquement justifiés et documentés. Les limites identifiées sur ce jeu de données synthétique constituent en elles-mêmes un résultat pédagogique. Elles illustrent l'importance de valider la pertinence statistique d'un indicateur avant de l'intégrer dans un système de détection, une précaution essentielle dans tout projet de data engineering appliqué à la fraude.

Bibliographie

- [1] MongoDB, Inc. MongoDB Manual, documentation officielle. <https://www.mongodb.com/docs/manual/>, consulté le 24 juillet 2026.
- [2] Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). *A Comprehensive Survey of Data Mining-based Fraud Detection Research*. arXiv:1009.6119.
- [3] Databricks. What is a Medallion Lakehouse Architecture? <https://www.databricks.com/glossary/medallion-architecture>, consulté le 25 juillet 2026.
- [4] Inmon, W. H. (2016). *Data Lake Architecture: Designing the Data Lake and Avoiding the Garbage Dump*. Technics Publications.
- [5] MongoDB, Inc. mongoimport, MongoDB Database Tools. <https://www.mongodb.com/docs/database-tools/mongoimport/>, consulté le 25 juillet 2026.
- [6] MongoDB, Inc. Aggregation Pipeline Documentation. <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>, consulté le 23 juillet 2026.
- [7] MongoDB, Inc. JSON Schema Validation. <https://www.mongodb.com/docs/manual/core/schema-validation/>, consulté le 23 juillet 2026.

Annexes : code des différentes couches

Cette annexe regroupe l'intégralité des scripts utilisés pour construire le pipeline, dans leur ordre d'exécution, ainsi que le lien vers le dépôt versionnant l'ensemble du projet.

Dépôt GitHub du projet : <https://github.com/miezansam/fraud-datalake-mongodb>

Ingestion brute, commandes mongoimport

Phase 1 : Ingestion brute (couche Bronze)

Principe

On charge les deux CSV ****tels quels****, sans aucune transformation. C'est le principe même d'un Data Lake : on conserve la donnée source intacte, y compris ses défauts (montants négatifs, valeurs manquantes), pour garder une traçabilité complète et pouvoir revenir en arrière si besoin.

Commandes (à exécuter dans l'invite de commandes Windows, PAS dans mongosh)

Adaptez les chemins vers l'endroit où sont stockés vos fichiers CSV (ex. `C:/Users/MIEZANSAM/Documents/`).

```
```bash
```

```
mongoimport --db fraud_datalake --collection raw_mobile_money --type csv --headerline --file "C:/Users/MIEZANSAM/Documents/transactions_mobile_money_100k.csv"
```

```
mongoimport --db fraud_datalake --collection raw_ventes --type csv --headerline --file "C:/Users/MIEZANSAM/Documents/ventes_commerce_abidjan_100k.csv"
```
```

- `--type csv --headerline` : indique que le fichier est un CSV et que la première ligne contient les noms de colonnes (mongoimport les utilise comme noms de champs).

- Chaque ligne du CSV devient un document MongoDB. Tous les champs sont importés en ****string**** à ce stade (normal pour du bronze — la conversion des types se fait en Silver).

Vérification après import

Dans `mongosh` :

```
```javascript
```

```
use fraud_datalake
```

```
db.raw_mobile_money.countDocuments() // doit afficher 100000
```

```
db.raw_ventes.countDocuments() // doit afficher 100000
```

```
db.raw_mobile_money.findOne()
db.raw_ventes.findOne()
'''
```

Vérifie que chaque `findOne()` correspond bien à la première ligne de données de ton CSV (hors en-tête).

## Point d'attention

Ne t'inquiète pas si tous les champs numériques (`montant\_fcfa`, `quantite`, etc.) apparaissent comme des chaînes de caractères (`"91784"` au lieu de `91784`) — c'est normal et attendu pour la couche bronze. La conversion de types se fait explicitement dans la couche Silver (script `02\_silver\_mobile\_money.js`), ce qui permet de documenter et justifier chaque transformation dans le rapport.

## Transformation Silver, flux Mobile Money (02\_silver\_mobile\_money.js)

```
// Phase 2 : Couche Silver : transactions_mobile_money
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/02_silver_mobile_money.js")
// =====
db.raw_mobile_money.aggregate([
 // 1. Conversion des types (bronze = tout en string)
 {
 $addFields: {
 dateHeureConvertie: {
 $dateFromString: {
 dateString: "$date_heure",
 format: "%Y-%m-%d %H:%M:%S",
 onError: null,
 onNull: null
 }
 },
 montantConverti: { $toDouble: "$montant_fcfa" },
 fraisConverti: {
 $cond: [
 { $in: ["$frais_fcfa", ["", null]] },
 null,
 { $toDouble: "$frais_fcfa" }
]
 },
 agentPropre: {
```

```

 $cond: [{ $in: ["$id_agent", ["", null]] }, null, "$id_agent"]
 },
 zoneBeneficiairePropre: {
 $cond: [{ $in: ["$zone_beneficiaire", ["", null]] }, null, "$zone_beneficiaire"]
 }
},
// 2. Structuration en documents imbriqués + flags qualité
{
 $project: {
 _id: "$id_transaction",
 source: { $literal: "mobile_money" },
 dateHeure: "$dateHeureConvertie",
 operateur: "$operateur",
 typeOperation: "$type_operation",
 expéditeur: {
 numero: "$expéditeur",
 zone: "$zone_expéditeur"
 },
 beneficiaire: {
 numero: "$beneficiaire",
 zone: "$zoneBeneficiairePropre"
 },
 montantFcfa: "$montantConverti",
 fraisFcfa: "$fraisConverti",
 idAgent: "$agentPropre",
 statut: "$statut",
 qualite: {
 montantValide: { $gt: ["$montantConverti", 0] },
 agentManquant: { $eq: ["$agentPropre", null] },
 zoneBeneficiaireManquante: { $eq: ["$zoneBeneficiairePropre", null] },
 dateValide: { $ne: ["$dateHeureConvertie", null] }
 }
 }
},
// 3. Écriture dans la collection silver (remplace si déjà existante)
{
 $merge: {
 into: "silver_mobile_money",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
}
]);

```



```

print("Transformation silver_mobile_money terminée.");
print("Nombre de documents : " + db.silver_mobile_money.countDocuments());
print("Documents avec montant invalide : " + db.silver_mobile_money.countDocuments({
"qualite.montantValide": false }));
print("Documents avec agent manquant : " + db.silver_mobile_money.countDocuments({
"qualite.agentManquant": true }));

```

## Transformation Silver, flux Ventes Commerce (03\_silver\_ventes.js)

```

// Phase 2 : Couche Silver : ventes_commerce
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/03_silver_ventes.js")
// =====
db.raw_ventes.aggregate([

// 1. Conversion des types (bronze = tout en string)
{
 $addFields: {
 dateConvertie: {
 $dateFromString: {
 dateString: "$date",
 format: "%Y-%m-%d",
 onError: null,
 onNull: null
 }
 },
 quantiteConvertie: { $toInt: "$quantite" },
 prixUnitaireConverti: { $toDouble: "$prix_unitaire" },
 montantConverti: { $toDouble: "$montant_fcfa" },
 vendeurPropre: {
 $cond: [{ $in: ["$vendeur", ["", null]] }, null, "$vendeur"]
 },
 zoneClientPropre: {
 $cond: [{ $in: ["$zone_client", ["", null]] }, null, "$zone_client"]
 }
 }
},

```

```
// 2. Structuration en documents imbriqués + flags qualité
```

```
{
 $project: {
 _id: "$id_transaction",
 source: { $literal: "ventes_commerce" },
 date: "$dateConvertie",
 produit: "$produit",
 quantite: "$quantiteConvertie",
 prixUnitaire: "$prixUnitaireConverti",
 montantFcfa: "$montantConverti",
 modePaiement: "$mode_paiement",
 vendeur: { nom: "$vendeurPropre" },
 client: { zone: "$zoneClientPropre" },
 qualite: {
 montantValide: { $gt: ["$montantConverti", 0] },
 vendeurManquant: { $eq: ["$vendeurPropre", null] },
 zoneClientManquante: { $eq: ["$zoneClientPropre", null] },
 dateValide: { $ne: ["$dateConvertie", null] },
 montantCoherent: {
 $eq: [
 "$montantConverti",
 { $multiply: ["$quantiteConvertie", "$prixUnitaireConverti"] }
]
 }
 }
 }
},
```

```
// 3. Écriture dans la collection silver
```

```
{
 $merge: {
 into: "silver_ventes",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
}
]);
print("Transformation silver_ventes terminée.");
print("Nombre de documents : " + db.silver_ventes.countDocuments());
print("Documents avec montant incohérent (qte x prix ≠ montant) : " +
db.silver_ventes.countDocuments({ "qualite.montantCoherent": false }));
print("Documents avec vendeur manquant : " + db.silver_ventes.countDocuments({
"qualite.vendeurManquant": true }));
```

**Création des index (04\_indexes.js)**

```
// Phase 2 (suite) : Création des index
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/04_indexes.js")
// =====
// --- silver_mobile_money ---
db.silver_mobile_money.createIndex({ "expediteur.numero": 1 });
db.silver_mobile_money.createIndex({ "beneficiaire.numero": 1 });
db.silver_mobile_money.createIndex({ "idAgent": 1 });
db.silver_mobile_money.createIndex({ "dateHeure": 1 });
db.silver_mobile_money.createIndex({ "expediteur.zone": 1, "beneficiaire.zone": 1 });
db.silver_mobile_money.createIndex({ "statut": 1 });

// --- silver_ventes ---
db.silver_ventes.createIndex({ "vendeur.nom": 1 });
db.silver_ventes.createIndex({ "client.zone": 1 });
db.silver_ventes.createIndex({ "date": 1 });
db.silver_ventes.createIndex({ "modePaiement": 1 });

print("Index créés.");
print("Index silver_mobile_money : " +
JSON.stringify(db.silver_mobile_money.getIndexes().map(i => i.name)));
print("Index silver_ventes : " + JSON.stringify(db.silver_ventes.getIndexes().map(i =>
i.name)));
```

**Calcul des features de fraude, couche Gold (05\_gold\_features.js)**

```
// Phase 3 : Couche Gold : features de détection de fraude
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/05_gold_features.js")
// =====

// Index supplémentaires nécessaires pour la corrélation 3.4 (jointure lourde)
db.silver_mobile_money.createIndex({ "montantFcfa": 1 });
db.silver_mobile_money.createIndex({ "beneficiaire.zone": 1 });

// -----
// 3.1 — Features par EXPÉDITEUR (numéro de téléphone)
// - nombre de transactions
// - montant total et moyen envoyé
// - écart-type des montants (variabilité suspecte)
// - nombre de zones distinctes utilisées (mobilité suspecte)
// - nombre de bénéficiaires distincts
// - taux d'échec
// -----
db.silver_mobile_money.aggregate([
 {
 $group: {
 _id: "$expediteur.numero",
 nbTransactions: { $sum: 1 },
 montantTotal: { $sum: "$montantFcfa" },
 montantMoyen: { $avg: "$montantFcfa" },
 montantEcartType: { $stdDevPop: "$montantFcfa" },
 zonesDistinctes: { $addToSet: "$expediteur.zone" },
 beneficiairesDistincts: { $addToSet: "$beneficiaire.numero" },
 nbEchecs: {
 $sum: { $cond: [{ $eq: ["$statut", "Échec"] }, 1, 0] }
 },
 premiereTransaction: { $min: "$dateHeure" },
 derniereTransaction: { $max: "$dateHeure" }
 }
 },
 {

```

```

$project: {
 _id: 1,
 nbTransactions: 1,
 montantTotal: 1,
 montantMoyen: { $round: ["$montantMoyen", 0] },
 montantEcartType: { $round: ["$montantEcartType", 0] },
 nbZonesDistinctes: { $size: "$zonesDistinctes" },
 nbBeneficiairesDistincts: { $size: "$beneficiairesDistincts" },
 tauxEchec: { $round: [{ $divide: ["$nbEchecs", "$nbTransactions"] }, 3] },
 premiereTransaction: 1,
 derniereTransaction: 1
}
},
{
 $merge: {
 into: "gold_features_expediteur",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
}
});

print("gold_features_expediteur créée : " + db.gold_features_expediteur.countDocuments() +
" expéditeurs distincts.");
// -----
// 3.2 — Features par AGENT (id_agent)
// - nombre de transactions traitées
// - taux d'échec par agent (un agent avec un taux anormalement
// élevé peut être impliqué dans des fraudes ou mal former)
// - montant total traité
// -----
db.silver_mobile_money.aggregate([
 { $match: { idAgent: { $ne: null } } },
 {
 $group: {
 _id: "$idAgent",
 nbTransactions: { $sum: 1 },
 montantTotal: { $sum: "$montantFcfa" },
 nbEchecs: {
 $sum: { $cond: [{ $eq: ["$statut", "Échec"] }, 1, 0] }
 }
 }
 },
 {

```

```

 $project: {
 _id: 1,
 nbTransactions: 1,
 montantTotal: 1,
 tauxEchec: { $round: [{ $divide: ["$nbEchecs", "$nbTransactions"] }, 3] }
 }
 },
 {
 $merge: {
 into: "gold_features_agent",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
 }
]);
print("gold_features_agent créée : " + db.gold_features_agent.countDocuments() + " agents distincts.");
// -----
// 3.3 — Transferts inter-zones rapprochés dans le temps
// Un même expéditeur qui envoie de l'argent depuis deux zones
// différentes en moins de 30 minutes est un signal fort
// (usurpation possible, compte partagé, etc.)
// -----
db.silver_mobile_money.aggregate([
 { $sort: { "expediteur.numero": 1, dateHeure: 1 } },
 {
 $group: {
 _id: "$expediteur.numero",
 transactions: {
 $push: { dateHeure: "$dateHeure", zone: "$expediteur.zone", idTransaction: "$_id" }
 }
 }
 },
 { $match: { $expr: { $gt: [{ $size: "$transactions" }, 1] } } },
 {
 $project: {
 _id: 1,
 paires: {
 $filter: {
 input: {
 $map: {
 input: { $range: [0, { $subtract: [{ $size: "$transactions" }, 1] }] },
 as: "i",
 in: {

```

```

 t1: { $arrayElemAt: ["$transactions", "$$i"] },
 t2: { $arrayElemAt: ["$transactions", { $add: ["$$i", 1] }] }
 }
 },
 as: "paire",
 cond: {
 $and: [
 { $ne: ["$$paire.t1.zone", "$$paire.t2.zone"] },
 {
 $lt: [
 { $abs: { $subtract: ["$$paire.t2.dateHeure", "$$paire.t1.dateHeure"] } },
 1800000 // 30 minutes en millisecondes
]
 }
]
 }
 }
}
},
{ $match: { $expr: { $gt: [{ $size: "$paires" }, 0] } } },
{
 $merge: {
 into: "gold_transferts_inter_zones_suspects",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
}
]);

print("gold_transferts_inter_zones_suspects créée : " +
db.gold_transferts_inter_zones_suspects.countDocuments() + " expéditeurs concernés.");

// -----
// 3.4 — Corrélation Mobile Money <-> Ventes commerce
// Ventes payées en Mobile Money le même jour, dans la même
// zone, pour un montant strictement identique à une transaction
// Mobile Money — signal de double-comptabilisation ou de
// détournement possible.

```

```
// -----
db.silver_ventes.aggregate([
 { $match: { modePaiement: "Mobile Money" } },
 {
 $lookup: {
 from: "silver_mobile_money",
 let: { montantVente: "$montantFcfa", zoneVente: "$client.zone", dateVente: "$date" },
 pipeline: [
 {
 $match: {
 $expr: {
 $and: [
 { $eq: ["$montantFcfa", "$$montantVente"] },
 { $eq: ["$beneficiaire.zone", "$$zoneVente"] },
 {
 $eq: [
 { $dateToString: { format: "%Y-%m-%d", date: "$dateHeure" } },
 { $dateToString: { format: "%Y-%m-%d", date: "$$dateVente" } }
]
 }
]
 }
 }
 }
]
 },
 as: "transactionsCorrespondantes"
 },
 { $match: { $expr: { $gt: [{ $size: "$transactionsCorrespondantes" }, 0] } } },
 {
 $project: {
 _id: 1,
 produit: 1,
 montantFcfa: 1,
 date: 1,
 client: 1,
 nbCorrespondances: { $size: "$transactionsCorrespondantes" }
 }
 },
 {
 $merge: {
 into: "gold_correlation_ventes_mobile_money",
 whenMatched: "replace",
 whenNotMatched: "insert"
 }
 }
])
```



```

 }
 }
});

print("gold_correlation_ventes_mobile_money créée : " +
db.gold_correlation_ventes_mobile_money.countDocuments() + " ventes corrélées.");

```

## Règles de scoring et génération des alertes (06\_fraud\_rules.js)

```

// =====
// Phase 4 : Règles de détection et scoring
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/06_fraud_rules.js")
//
// Principe : on calcule des seuils (fixes ou statistiques selon la pertinence
// pour ce jeu de données) sur les features Gold, on marque les entités "à
// risque", puis on calcule un score de risque par transaction en combinant :
// - anomalies propres à la transaction (montant invalide)
// - anomalies propres à l'expéditeur (vélocité, variabilité, multi-bénéficiaires)
// - anomalies propres à l'agent (taux d'échec élevé, calculé par percentile 95)
//
// ⚠ Les seuils (2 points, 1.5x, 0.3, etc.) sont volontairement
// simples et doivent être justifiés/ajustés dans votre rapport
// ce n'est pas un modèle ML, mais un système de règles explicable.
// =====

// --- 4.1 — Calcul des seuils statistiques (percentile 95) ---

// Note importante : le percentile 95 du nombre de transactions (et du nombre
// de bénéficiaires distincts) par expéditeur tombe sur 1 dans ce jeu de données,
// car 98,7% des expéditeurs n'envoient qu'une seule transaction sur toute la
// période (voir analyse de distribution dans le rapport). Utiliser ce percentile
// comme seuil flaguerait presque tout le monde dès la 2e transaction, ce qui n'a
// aucun sens métier. On fixe donc des seuils manuels plus réalistes : les
// expéditeurs ayant 2 transactions ou plus représentent déjà le groupe le plus
// actif (1,3% des expéditeurs), on retient donc ce seuil comme signal faible.
const seuilTransactions = 2;
const seuilBeneficiaires = 2;
print("Seuil fixe (justifié) nombre de transactions par expéditeur : " + seuilTransactions);
print("Seuil fixe (justifié) nombre de bénéficiaires distincts : " + seuilBeneficiaires);

```

```

const statsAgent = db.gold_features_agent.aggregate([
 { $match: { nbTransactions: { $gte: 30 } } },
 {
 $group: {
 _id: null,
 p95TauxEchec: { $percentile: { input: "$tauxEchec", p: [0.95], method: "approximate" } }
 }
 }
]).toArray()[0];

const seuilTauxEchecAgent = statsAgent.p95TauxEchec[0];
print("Seuil P95 taux d'échec agent (agents avec >= 30 transactions) : " +
seuilTauxEchecAgent);

// --- 4.2 — Marquage des expéditeurs à risque ---

db.gold_features_expéditeur.aggregate([
 {
 $addFields: {
 veloticeSuspecte: { $gte: ["$nbTransactions", seuilTransactions] },
 multiBeneficiairesSuspecte: { $gte: ["$nbBeneficiairesDistincts", seuilBeneficiaires] },
 variabiliteSuspecte: {
 $and: [
 { $gte: ["$nbTransactions", 5] },
 {
 $gt: [
 { $cond: [{ $eq: ["$montantMoyen", 0] }, 0, { $divide: ["$montantEcartType",
"$montantMoyen"] }] }],
 1.5
]
 }
]
 },
 },
 },
 // ⚠ Le taux d'échec n'est fiable statistiquement qu'avec un minimum
 // d'observations. Sur un échantillon d'une seule transaction, le taux
 // vaut mécaniquement 0% ou 100% sans que cela soit un vrai signal.
 // On exige donc au moins 5 transactions avant de considérer ce critère
 // (ce qui, vu le maximum de 3 transactions par expéditeur observé dans
 // ce jeu de données, désactive de fait ce critère au niveau expéditeur
 // c'est un résultat honnête à documenter, pas un bug).

```

```

tauxEchecEleve: {
 $and: [{ $gte: ["$nbTransactions", 5] }, { $gt: ["$tauxEchec", 0.3] }]
}
},
{
 $addFields: {
 scoreRisque: {
 $sum: [
 { $cond: ["$veloticeSuspecte", 1, 0] },
 { $cond: ["$multiBeneficiairesSuspect", 1, 0] },
 { $cond: ["$variabiliteSuspecte", 2, 0] },
 { $cond: ["$tauxEchecEleve", 1, 0] }
]
 }
 },
 { $merge: { into: "gold_features_expediteur", whenMatched: "merge", whenNotMatched:
"discard" } }
]);

print("Expéditeurs à risque (score > 0) : " + db.gold_features_expediteur.countDocuments({
scoreRisque: { $gt: 0 } }));

// --- 4.3 — Marquage des agents à risque ---

db.gold_features_agent.aggregate([
{
 $addFields: {
 tauxEchecSuspect: {
 $and: [{ $gte: ["$nbTransactions", 30] }, { $gt: ["$tauxEchec", seuilTauxEchecAgent] }]
 }
 },
 { $addFields: { scoreRisque: { $cond: ["$tauxEchecSuspect", 3, 0] } } },
 { $merge: { into: "gold_features_agent", whenMatched: "merge", whenNotMatched:
"discard" } }
]);

print("Agents à risque (score > 0) : " + db.gold_features_agent.countDocuments({
scoreRisque: { $gt: 0 } }));

```

```
// --- 4.4 — Génération des alertes au niveau transaction ---
```

```
db.silver_mobile_money.aggregate([
 {
 $lookup: {
 from: "gold_features_expediteur",
 localField: "expediteur.numero",
 foreignField: "_id",
 as: "profilExpediteur"
 }
 },
 { $unwind: { path: "$profilExpediteur", preserveNullAndEmptyArrays: true } },
 {
 $lookup: {
 from: "gold_features_agent",
 localField: "idAgent",
 foreignField: "_id",
 as: "profilAgent"
 }
 },
 { $unwind: { path: "$profilAgent", preserveNullAndEmptyArrays: true } },
 {
 $addFields: {
 scoreTransaction: { $cond: [{ $eq: ["$qualite.montantValide", false] }, 3, 0] },
 scoreExpediteur: { $ifNull: ["$profilExpediteur.scoreRisque", 0] },
 scoreAgent: { $ifNull: ["$profilAgent.scoreRisque", 0] }
 }
 },
 {
 $addFields: {
 scoreTotal: { $add: ["$scoreTransaction", "$scoreExpediteur", "$scoreAgent"] }
 }
 },
 { $match: { scoreTotal: { $gte: 2 } } },
 {
 $project: {
 _id: 1,
 dateHeure: 1,
 expediteur: 1,
 beneficiaire: 1,
 montantFcfa: 1,
 idAgent: 1,
 statut: 1,
 scoreTotal: 1,
 }
 }
])
```

```

 motifs: {
 $concatArrays: [
 { $cond: [{ $eq: ["$qualite.montantValide", false] }, ["Montant invalide (≤ 0)"], []] },
 { $cond: ["$profilExpediteur.veloticeSuspecte", ["Fréquence de transactions anormalement élevée pour cet expéditeur"], []] },
 { $cond: ["$profilExpediteur.multiBeneficiairesSuspect", ["Nombre inhabituel de bénéficiaires distincts"], []] },
 { $cond: ["$profilExpediteur.variabiliteSuspecte", ["Montants très irréguliers pour cet expéditeur"], []] },
 { $cond: ["$profilAgent.tauxEchecSuspect", ["Agent avec un taux d'échec anormalement élevé"], []] }
]
 }
 },
 { $merge: { into: "alertes_fraude", whenMatched: "replace", whenNotMatched: "insert" } }
]);

print("-----");
print("Total alertes générées : " + db.alertes_fraude.countDocuments());
print("Répartition par score :");
db.alertes_fraude.aggregate([
 { $group: { _id: "$scoreTotal", nb: { $sum: 1 } } },
 { $sort: { _id: 1 } }
]).forEach(doc => print(" score " + doc._id + " : " + doc.nb + " alertes"));

// --- 4.5 — Niveau de sévérité (pour un rapport plus nuancé) ---
// score 2 : à surveiller (signal faible, souvent isolé)
// score 3 à 4 : suspect (un critère fort ou une combinaison de critères faibles)
// score 5 et + : prioritaire (plusieurs critères forts combinés)

db.alertes_fraude.updateMany(
 { scoreTotal: 2 },
 { $set: { niveauRisque: "A_SURVEILLER" } }
);
db.alertes_fraude.updateMany(
 { scoreTotal: { $gte: 3, $lte: 4 } },
 { $set: { niveauRisque: "SUSPECT" } }
);
db.alertes_fraude.updateMany(
 { scoreTotal: { $gte: 5 } },
 { $set: { niveauRisque: "PRIORITAIRE" } }
);

```

```

print("-----");
print("Répartition par niveau de sévérité :");
db.alertes_fraude.aggregate([
 { $group: { _id: "$niveauRisque", nb: { $sum: 1 } } },
 { $sort: { nb: -1 } }
]).forEach(doc=> print(" " + doc._id + " : " + doc.nb + " alertes"));

```

### Validation de schéma sur la couche Silver (07\_validator\_silver.js)

```

// =====
// Phase 2 (complément) — Validation de schéma sur silver_mobile_money
// À exécuter dans mongosh, après avoir fait : use fraud_datalake
// Usage : load("scripts/07_validator_silver.js")
//
// Principe : on applique un $jsonSchema à la collection déjà peuplée,
// en validationLevel "moderate" (ne s'applique qu'aux futures
// insertions/modifications, ne rejette pas les documents déjà présents).
// Cela garantit qu'à partir de maintenant, plus aucun document mal
// structuré ne peut être inséré silencieusement dans la couche Silver,
// ce qui complète les flags de qualité calculés au moment du chargement.
// =====

db.runCommand({
 collMod: "silver_mobile_money",
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["_id", "source", "expediteur", "beneficiaire", "montantFcfa", "statut", "qualite"],
 properties: {
 _id: { bsonType: "string" },
 source: { bsonType: "string", enum: ["mobile_money"] },
 dateHeure: { bsonType: ["date", "null"] },
 operateur: { bsonType: "string" },
 typeOperation: { bsonType: "string" },
 expediteur: {
 bsonType: "object",
 required: ["numero"],
 properties: {
 numero: { bsonType: "string" },
 zone: { bsonType: ["string", "null"] }
 }
 }
 }
 }
 },

```

```

 beneficiaire: {
 bsonType: "object",
 required: ["numero"],
 properties: {
 numero: { bsonType: "string" },
 zone: { bsonType: ["string", "null"] }
 }
 },
 montantFcfa: { bsonType: "double" },
 fraisFcfa: { bsonType: ["double", "null"] },
 idAgent: { bsonType: ["string", "null"] },
 statut: { bsonType: "string" },
 qualite: {
 bsonType: "object",
 properties: {
 montantValide: { bsonType: "bool" },
 agentManquant: { bsonType: "bool" },
 zoneBeneficiaireManquante: { bsonType: "bool" },
 dateValide: { bsonType: "bool" }
 }
 }
 },
 validationLevel: "moderate",
 validationAction: "error"
});

print("Valdateur JSON Schema appliqué à silver_mobile_money (validationLevel:
moderate).");

// Test : une tentative d'insertion d'un document non conforme doit échouer
try {
 db.silver_mobile_money.insertOne({ _id: "TEST_INVALIDE", source: "mobile_money" });
 print("ERREUR : le document non conforme a été accepté, le validateur ne fonctionne pas.");
} catch (e) {
 print("Test réussi : le validateur rejette bien un document incomplet.");
 print("Message : " + e.message);
}

```